Analisi Tecnica Completa: Regression Learner per V2G

Autore: Analisi generata da Antigravity AI
Data: Marzo 2026
Progetto: V2G-with-Regression-Learner-and-LSTM-NN
Script di riferimento: `RegressionLearner.m`

Indice

1. Contesto Applicativo: Vehicle-to-Grid (V2G)
2. Il Problema: Previsione del Consumo Energetico
3. Approccio Tabular Regression vs LSTM
4. Il Modello Ensemble: Gradient Boosted Trees
5. Struttura del Dataset
6. Architettura del Codice
7. Pipeline Dati: dal Raw al Modello
8. Costruzione della Matrice dei Regressori
9. Problema del Persistence Shortcut e Soluzioni Sperimentate
10. Valutazione e Metriche
11. Risultati Sperimentali Comparativi
12. Analisi Critica e Confronto con LSTM
13. Possibili Sviluppi Futuri

---

1. Contesto Applicativo: Vehicle-to-Grid (V2G)

Cos'è il V2G?

La tecnologia **Vehicle-to-Grid (V2G)** è un paradigma emergente nella gestione dell'energia distribuita in cui i veicoli elettrici (EV) non sono semplici consumatori di energia, ma diventano **partecipanti attivi e bidirezionali della rete elettrica**.

In un sistema V2G il veicolo elettrico può:

- **Caricarsi** dalla rete durante le ore di bassa domanda (notte, festivi) quando l'energia è abbondante e a costo ridotto.
- **Cedere energia alla rete** durante i picchi di domanda (ore mattutine e serali nei giorni feriali), contribuendo alla stabilizzazione della frequenza e della tensione di rete.

Questo crea un ecosistema bidirezionale che beneficia simultaneamente l'utente finale (riduzione dei costi energetici attraverso l'arbitraggio), il gestore della rete (flessibilità e bilanciamento), e il sistema elettrico nel suo complesso (maggiore integrazione delle fonti rinnovabili intermittenti).

Perché la Previsione del Consumo è Fondamentale in V2G?

Per ottimizzare le strategie di carica/scarica V2G, è indispensabile **conoscere in anticipo** il profilo di consumo energetico delle zone urbane in cui gli EV sono connessi. Questo consente al sistema di:

1. **Pianificare la carica** degli EV nelle fasce orarie a basso carico di rete.
2. **Programmare la restituzione** di energia nelle fasce di picco, massimizzando il beneficio alla rete.
3. **Evitare di sovraccaricare la rete** in momenti già critici.
4. **Massimizzare l'arbitraggio sul prezzo dell'energia** (acquistare nelle ore off-peak, vendere nelle ore on-peak).

5. **Garantire la disponibilità del veicolo** per l'utente (non scaricare sotto la soglia di autonomia garantita).

Un modello predittivo accurato del consumo energetico è quindi il **nucleo computazionale** di qualsiasi sistema V2G intelligente. L'errore del modello si traduce direttamente in perdita economica, inefficienza gestionale o discomfort per l'utente.

Il Contesto Geografico: Roma, 2023

Il progetto lavora su dati reali di consumo energetico di **zone di distribuzione della città di Roma** per l'anno 2023. I dati hanno risoluzione **half-hourly** (30 minuti), producendo 48 campioni per giorno per zona.

ID	Nome	Carattere urbanistico
8	Zone_1016_Anagnina	Residenziale/Commerciale periferia sud
9	Zone_214_Trieste	Residenziale denso, nord Roma (**zona analizzata**)
10	Zone_2004_Della Vittoria/Tor di Quinto	Misto commerciale-residenziale
11	Zone2002_Tor di Quinto	Residenziale

I dati **non coprono l'intero anno 2023 in modo continuo**, ma sono organizzati in 4 cluster temporali con gap tra uno e l'altro — questo è un vincolo fondamentale dell'intero progetto:

```
Cluster 1: [15 Feb 2023] → [28 Feb 2023]   ≈ 14 giorni  (inverno)
Cluster 2: [31 Mag 2023] → [15 Giu 2023]   ≈ 15 giorni  (primavera/estate)
Cluster 3: [12 Lug 2023] → [25 Lug 2023]   ≈ 14 giorni  (estate piena)
Cluster 4: [27 Set 2023] → [10 Ott 2023]   ≈ 14 giorni  (autunno)
```

In totale circa **57 giorni di dati**, distribuiti su 4 stagioni diverse. Questa scarsità di dati è il principale fattore limitante di tutto il progetto.

---

2. Il Problema: Previsione del Consumo Energetico

Tipo di Problema

Il problema è classificabile come **regressione su serie temporale multivariata**: dato un insieme di osservazioni storiche fino al tempo *t-1*, prevedere il consumo energetico al tempo *t* (il passo successivo di 30 minuti).

- **Input:** vettori di feature descrittive degli step temporali precedenti (meteo, ora, giorno, consumo storico)
- **Output:** valore scalare — il consumo energetico in kWh al prossimo step
- **Orizzonte di previsione:** 1 step (30 minuti) — previsione one-step-ahead

Feature Selezionate

Il modello usa **9 predittori** suddivisi in tre categorie funzionali:

Variabili meteorologiche (esogene — indipendenti dalla zona):

- `precipprob` — Probabilità di precipitazione (%)
- `temp` — Temperatura dell'aria (°C)
- `windspeed` — Velocità del vento (km/h)

Variabile calendario:

- `holiday_indicator` — Flag binario (0 = giorno ordinario, 1 = festivo/domenica)

Variabili cicliche (encoding sinusoidale dell'ora e del giorno):

- `hour_sin = sin(2π × ora / 24)`
- `hour_cos = cos(2π × ora / 24)`
- `day_sin = sin(2π × dow / 7)` (dow = day of week)
- `day_cos = cos(2π × dow / 7)`

> ***Nota sull'encoding ciclico:*** *Usare sin/cos anziché il numero diretto dell'ora risolve il problema della discontinuità: ora 23 e ora 0 sono numericamente "lontane" (23 vs 0) ma temporalmente contigue. Con sin/cos, sono rappresentate come punti adiacenti su un cerchio unitario. Lo stesso vale per lunedì/domenica nel ciclo settimanale.*

Variabile autoregressive (endogena):

- `AAC_energy` — Il consumo energetico stesso in kWh, che funge da autoregressor. Usato con lag 1..48.

> ***Attenzione:*** *Nella versione corrente del codice (`_exog`), `AAC_energy` è stato rimosso dai predittori per risolvere il persistence shortcut — vedi Sezione 9.*

---

3. Approccio Tabular Regression vs LSTM

Differenza Fondamentale di Paradigma

L'LSTM (vedi `analisi_progetto_LSTM.md`) riceve i dati come **sequenze temporali 3D** e mantiene uno stato interno ricorrente che evolve nel tempo: questo gli permette di "ricordare" pattern su lunghe finestre temporali in modo implicito.

Il **Regression Learner** adotta invece un approccio **tabular (a tabella piatta)**:

```
LSTM:   Input = [sequenza di T × 9 feature] → mantiene stato ricorrente
Regr.:  Input = [vettore piatto di 48 × 9 = 432 colonne] → stateless
```

Nel Regression Learner, tutte le informazioni contestuali vengono "schiacciato" in un'unica riga della tabella di addestramento. Ogni riga corrisponde a un istante temporale *t* e contiene:

- I valori delle 9 (o 8) feature ai 48 step precedenti (lag 1..48)
- Il target: il consumo al tempo *t*

Vantaggi della tabular regression:

- Semplicità implementativa: nessuna architettura ricorrente
- Interpretabilità: feature importance direttamente leggibile
- Velocità di training: minuti invece di decine di minuti
- Non richiede padding o gestione di sequenze di lunghezze diverse

Svantaggi:

- Non cattura dipendenze temporali in modo implicito — ogni riga è indipendente
- Con finestre da 48 lag e 9 feature → **432 colonne** di input, molte delle quali altamente correlate
- Rischio elevato di overfitting con pochi dati (2064 training samples, 432 feature)
- Il problema del "persistence shortcut" (vedi Sezione 9)

---

4. Il Modello Ensemble: Gradient Boosted Trees

Cos'è il Gradient Boosting?

Il modello predefinito in `RegressionLearner.m` è un **ensemble di alberi di regressione** addestrati con l'algoritmo **Least Squares Boosting (LSBoost)**, variante del Gradient Boosting per regressione con perdita L2.

L'idea del boosting è addestrare alberi **sequenzialmente**: ogni nuovo albero viene addestrato per correggere gli errori del precedente. Formalmente:

```
F_0(x) = media(y)
F_m(x) = F_{m-1}(x) + η · h_m(x)
```

dove:

- `F_m(x)` è il predittore cumulativo dopo `m` iterazioni
- `η` è il learning rate (0.05 nel codice)
- `h_m(x)` è il nuovo albero addestrato sui **residui** del modello precedente

Il termine "gradient" deriva dal fatto che i residui al passo *m* sono esattamente il gradiente negativo della loss MSE rispetto alle predizioni attuali.

Configurazione dell'Ensemble in `RegressionLearner.m`

```
mdl = fitrensemble(XTrain, yTrain, ...
    "Method",           "LSBoost", ...
    "NumLearningCycles", 300, ...      % 300 alberi nell'ensemble
    "LearnRate",        0.05, ...      % learning rate basso = più robusto
    "Learners",         templateTree("MaxNumSplits", 6));  % alberi piccoli (profondità ~3-
4)
```

Scelte di design:

- **300 alberi:** Numero sufficiente per convergere con learning rate 0.05. Più alberi = più stabile ma più lento.
- **Learning rate 0.05:** Valore basso → ogni singolo albero contribuisce poco → il modello finale è più robusto e generalizza meglio. Un learning rate alto (es. 0.3) converge più in fretta ma overfits.
- **MaxNumSplits = 6:** Ogni albero ha al massimo 6 split interni (profondità ≈ 3). Alberi "stumps" (superficiali) sono ideali per boosting: catturano pattern semplici che l'ensemble combina in predizioni complesse.

Altri Modelli Disponibili

`RegressionLearner.m` implementa 5 modelli selezionabili tramite la costante `modelType`:

`modelType`	Funzione MATLAB	Caratteristiche
`"ensemble"`	`fitrensemble` (LSBoost)	Robusto, buon bias-variance tradeoff
`"tree"`	`fitrtree`	Interpretabile, veloce, soffre overfitting
`"svm"`	`fitrsvm` (RBF kernel)	Potente su feature non lineari, lento con 432 feature
`"gpr"`	`fitrgp` (squared-exp kernel)	Fornisce incertezza, molto lento su dataset grandi
`"linear"`	`fitrlinear` (Lasso)	Velocissimo, interpretabile, limitato su non-linearità

---

5. Struttura del Dataset

File Sorgente

Il dataset è composto da tre sorgenti distinte fuse da `loadZoneData.m`:

Dati di consumo per zona (`/Dati Estratti (from Condivisione)/`):

- `Zone214_new.mat` → variabili: `AAC_energy` (tabella), `time_vector` (datetime)
- `Zone1016_new.mat` , `Zone2004_new.mat` , `Zone2002_new.mat` → stessa struttura

Ogni file contiene un vettore temporale e i consumi in kWh a risoluzione 30 minuti su un anno.

Dati meteorologici (`/Gabriele Datas/metero_year_hh.mat`):

- Tabella `meteo_year_hh` con colonne: `temp` , `precipprob` , `windspeed` (e altri campi)
- Frequenza: 30 minuti, sincronizzata con i dati di consumo.

Calendario Festività (`/Gabriele Datas/holidays.mat`):

- Vettore `vacanze` (0 = giorno ordinario, 1 = festivo/domenica)
- Copre l'intero anno.

Struttura della Tabella Unificata

Dopo `loadZoneData` , si ottiene una singola tabella MATLAB con le seguenti colonne:

Colonna	Tipo	Descrizione
`time_vector`	datetime	Timestamp a 30 min
`temp`	double	Temperatura (°C)
`precipprob`	double	Probabilità precipitazioni (%)
`windspeed`	double	Velocità vento (km/h)
`holiday_indicator`	double	0/1 flag festivo
`AAC_energy`	double	Consumo energetico (kWh)

Dopo la fase di feature engineering, vengono aggiunte:

Colonna	Formula
`hour_sin`	`sin(2π × hour(t) / 24)`
`hour_cos`	`cos(2π × hour(t) / 24)`
`day_sin`	`sin(2π × dow(t) / 7)`
`day_cos`	`cos(2π × dow(t) / 7)`

Distribuzione dei Dati (Zona 9 — Trieste)

```
Dataset completo:  57 giorni × 48 campioni = 2736 righe
Training set:      ~43 giorni → 2064 campioni (dopo lag matrix)
```

```
Validation set:     2 giorni → 96 campioni
Test set:           2 giorni → 96 campioni
```

I 2 giorni di validation e i 2 di test vengono rimossi dal dataset prima del training per prevenire il data leakage.

---

6. Architettura del Codice

```
V2G-with-Regression-Learner-and-LSTM-NN/
├── RegressionLearner.m              ← Entry point: pipeline completa
├── LSTM.m                           ← Modello alternativo (reti ricorrenti)
├── LSTM_Predictions.m               ← Previsione cross-zona
└── scripts/
    ├── loadZoneData.m               ← Caricamento e fusione dati zone
    ├── selectRepresentativeDays.m   ← Selezione automatica giorni val/test
    ├── splitTrainValTest.m          ← Split dataset train/val/test
    ├── normalizeZScore.m            ← Normalizzazione z-score anti-leakage
    ├── createRegressionLags.m       ← Costruzione matrice regressori (lag matrix)
    ├── computeMetrics.m             ← Calcolo RMSE, R², MAE, MAPE
    ├── saveModels.m                 ← Salvataggio persistente modello
    └── plotRegressionResults.m      ← Visualizzazione real vs predicted
```

Descrizione di Ogni Script

`RegressionLearner.m` — Entry Point

Script principale che orchestra l'intera pipeline. Sezioni principali (demarcate da `%%`):

Sezione	Righe (appross.)	Funzione
Data Loading	1–55	Carica dati zona, aggiunge feature cicliche
Constants	53–72	`numLags`, `modelType`, `predictors`, `target`
Data Preparation	74–87	Split + normalizzazione z-score
Lag Matrix Creation	89–119	Chiama `createRegressionLags` per tutti e 3 i set
Model Training	`switch modelType`	Addestra il modello selezionato
Predictions	192–201	`predict(mdl, X)` su tutti e 3 i set
Metrics	203–206	`computeMetrics` per Train/Val/Test
Model Saving	208–246	`saveModels` in cartella `Sessioni/`
Final Summary	248–276	Stampa tabella metriche nel Command Window
Plots	278–360	Time-series, scatter, residual analysis, hourly breakdown
Persistence Baseline	362–378	Confronto con y(t)=y(t-1)

`loadZoneData.m` — Caricamento Dati

```
function dataOut = loadZoneData(projectRoot, zoneId)
```

Seleziona il file `.mat` corretto in base a `zoneId`, carica i dati di consumo, meteo e festività, e li fonde in un'unica tabella MATLAB. La tabella meteo (`meteo_year_hh`) funge da base: i consumi e le festività vengono aggiunti come colonne aggiuntive.

`selectRepresentativeDays.m` — Selezione Giorni Val/Test

```
function [valDays, testDays] = selectRepresentativeDays(datas)
```

Algoritmo in 4 fasi:

1. **Clustering temporale:** individua i 4 cluster separati da gap > 7 giorni.
2. **Riassunto giornaliero:** per ogni giorno calcola consumo totale (somma), temperatura media, precipitazione media, velocità vento media.
3. **Selezione best day per cluster:** per ogni cluster seleziona il giorno normale (non domenica) e la domenica più "tipici". "Tipico" = filtro a due stadi: prima i 5 giorni più vicini alla mediana energetica del cluster, poi tra questi il giorno con meteo più vicino alla mediana meterologica del cluster.
4. **Vincolo di contiguità:** il giorno selezionato deve avere il giorno precedente (t-1) disponibile nel dataset (necessario per i lag della matrice regressori).

Risultato: 2 giorni per validation (1 normale + 1 domenica) e 2 giorni per test (1 domenica + 1 normale), da cluster stagionali distinti.

`splitTrainValTest.m` — Split Dataset

```
function [training, validation, test] = splitTrainValTest(inputTable, validationDays,
testDays)
```

Rimuove i giorni val/test dal dataset principale usando `ismember`. Il training set contiene tutto il resto. L'operazione è puramente su maschere booleane sui timestamps.

`normalizeZScore.m` — Normalizzazione Anti-Leakage

```
function [trainNorm, secondNorm, normParams] = normalizeZScore(training, second,
columnsToNorm)
```

Principio anti-data-leakage fondamentale: la media μ e la deviazione standard σ vengono calcolate **esclusivamente sul training set**. Questi stessi parametri vengono poi applicati ai set di validazione e test:

```
x_norm = (x - μ_train) / σ_train
```

Se normalizzassimo con statistiche sull'intero dataset, il modello "vedrebbe" indirettamente informazioni future durante l'addestramento, gonfiando falsamente le metriche. Gestisce anche il caso σ = 0 (feature costante) impostando σ = 1 per evitare divisioni per zero.

Il struct `normParams` contiene `normParams.AAC_energy.mean` e `normParams.AAC_energy.std` per denormalizzare le predizioni in kWh.

`createRegressionLags.m` — Costruzione Matrice Regressori

La funzione più critica per il Regression Learner — vedi Sezione 8 per la spiegazione dettagliata.

`computeMetrics.m` — Calcolo Metriche

```
function indicators = computeMetrics(yPredNorm, tDataNorm, params, target)
```

Riceve predizioni e target **normalizzati**, li denormalizza internamente e calcola:

- **R^2** = 1 - SS_res / SS_tot
- **RMSE** = √mean(errors^2)
- **MSE** = mean(errors^2)
- **MAE** = mean(|errors|)
- **MAPE** = mean(|errors[mask]| / |tData[mask]|) × 100, dove `mask = |tData| > 1e-3` per escludere valori prossimi a zero (protezione da esplosione numerica)

`saveModels.m` — Salvataggio Persistente

```
function saveModels(rootFolder, models, varName, currentNet)
```

Organizza i modelli salvati in una struttura gerarchica:

```
Sessioni/
└── 2026_03_06/
    └── Models_2026_03_06.mat   ← struct 'models' con tutti i modelli del giorno
```

Se il file `.mat` del giorno esiste già, il nuovo modello viene **aggiunto** alla struct esistente senza sovrascrivere i modelli precedenti. Usa `-v7.3` per supportare file > 2GB.

`plotRegressionResults.m` — Visualizzazione Time-Series

```
function figureRegressionLearner = plotRegressionResults(yPredNorm, yNorm, params, target,
timeVector)
```

Riceve predizioni e target normalizzati, li denormalizza e genera un grafico confronto real vs predicted con:

- Separatori verticali tra i giorni di test
- Etichette data sull'asse X
- Legenda real/predicted con colori nero/rosso

---

7. Pipeline Dati: dal Raw al Modello

La pipeline in `RegressionLearner.m` segue esattamente questi 7 passi sequenziali:

Passo 1 — Caricamento e Feature Engineering

```
datas = loadZoneData(root, zoneId);
datas.hour_sin = sin(2 * pi * hour(datas.time_vector) / 24);
datas.hour_cos = cos(2 * pi * hour(datas.time_vector) / 24);
```

```
datas.day_sin  = sin(2 * pi * day(datas.time_vector, "dayofweek") / 7);
datas.day_cos  = cos(2 * pi * day(datas.time_vector, "dayofweek") / 7);
```

Le feature cicliche vengono calcolate **prima** dello split per mantenere consistenza. Poiché sono funzioni deterministiche del timestamp (non dipendono dalla statistica dei dati), non introducono data leakage.

Passo 2 — Selezione Giorni Rappresentativi

```
[validationDays, testDays] = selectRepresentativeDays(datas);
```

Seleziona automaticamente 2+2 giorni rappresentativi usando il clustering temporale e la selezione energetica/meteorologica descritta in Sezione 6.

Passo 3 — Split Train / Validation / Test

```
[training, validation, test] = splitTrainValTest(datas, validationDays, testDays);
```

I giorni val/test vengono estratti. Il training set contiene tutto il resto (~43 giorni).

Passo 4 — Normalizzazione Z-Score

```
[trainingNorm, testNorm,       normParams] = normalizeZScore(training, test,
columnsToNormalize);
[~,            validationNorm, ~         ] = normalizeZScore(training, validation,
columnsToNormalize);
```

Due chiamate separate: la seconda utilizza ancora le statistiche del training set (passato come primo argomento), garantendo zero leakage. I `normParams` vengono salvati nel modello per la successiva denormalizzazione delle predizioni.

Nota critica: con la configurazione `_exog` attuale (solo feature esogene), `columnsToNormalize` include `AAC_energy` (target) ma **non** le feature AAC_energy come predictors.

Passo 5 — Costruzione Matrice dei Regressori

```
trainLag = createRegressionLags(trainingNorm, numLags, predictors, char(target));
valLag   = createRegressionLags(validationNorm, numLags, predictors, char(target));
testLag  = createRegressionLags(testNorm, numLags, predictors, char(target));
```

Questo è il passo più critico — vedi Sezione 8 per la spiegazione completa dell'algoritmo.

Passo 6 — Estrazione X e y dalla Tabella

```
featureCols = trainLag.Properties.VariableNames;
featureCols = featureCols(~strcmp(featureCols, "time_vector") & ~strcmp(featureCols,
char(target)));
```

```
XTrain = table2array(trainLag(:, featureCols));   % [2064 × 384] (config exog)
yTrain = trainLag.(char(target));                 % [2064 × 1]
```

Passo 7 — Training e Predizione

```
mdl = fitrensemble(XTrain, yTrain, ...);
yTestPred = predict(mdl, XTest);
```

Le predizioni sono ancora nello spazio normalizzato. La denormalizzazione avviene in `computeMetrics` e nella sezione plot.

---

8. Costruzione della Matrice dei Regressori

L'Algoritmo di `createRegressionLags`

La funzione trasforma una tabella temporale in una matrice tabulare dove ogni riga rappresenta un istante *t* e contiene i valori delle feature ai *lag* step precedenti.

Struttura dell'output:

Per `numLags = 48` e `predictors` = {P_1, P_2, ..., P_8} (8 feature nella config exog):

```
Colonne output [numLags × numPredictors + 1 colonne totali]:
  P1_t_48, P2_t_48, ..., P8_t_48,    ← lag più lontano (t-48 = 24h fa)
  P1_t_47, P2_t_47, ..., P8_t_47,
  ...
  P1_t_1,  P2_t_1,  ..., P8_t_1,     ← lag immediato (t-1 = 30 min fa)
  AAC_energy                           ← target (consumo al tempo t)
  time_vector                          ← timestamp (aggiunto separatamente)
```

Con 8 feature × 48 lag = **384 colonne di input** per la config `_exog`, o 9 × 48 = **432 colonne** nella config originale con AAC_energy.

Gestione della contiguità temporale:

L'algoritmo gestisce i gap temporali nel dataset in modo rigoroso:

```
1. Calcola i giorni unici presenti nel dataset
2. Per ogni coppia (giorno corrente, giorno precedente):
   - Verifica che siano giorni consecutivi nel calendario
   - Se sì → il giorno è "valido" per il lagging
   - Se no → il giorno viene saltato (la finestra di contesto attraverserebbe un gap)
3. Per ogni giorno valido:
   - Concatena (giorno_precedente, giorno_corrente) in un blocco da 96 righe
   - Scorre il blocco da riga (numLags+1) in poi
   - Per ogni riga t: costruisce il vettore [feature[t-48] ... feature[t-1], target[t]]
```

Questo approccio garantisce che **nessuna sequenza di lag attraversi mai un gap temporale**, evitando l'introduzione di pattern spuri nella matrice di training.

Nomenclatura delle colonne:

Il nome di ogni colonna è generato come `sprintf("%s_t_%d", predictor, lag)`:

- `temp_t_48` → temperatura 24 ore fa
- `precipprob_t_1` → probabilità pioggia 30 minuti fa
- `hour_sin_t_24` → encoding ciclico dell'ora 12 ore fa

Risultati numerici (Zona 9, config exog):

```
Training:
  Total days: 50  |  Valid consecutive days: 43  |  Total rows: 2064

Validation:
  Total days: 4   |  Valid consecutive days: 2   |  Total rows: 96

Test:
  Total days: 4   |  Valid consecutive days: 2   |  Total rows: 96
```

Il gap tra "Total days" e "Valid consecutive days" dipende dalla struttura a cluster del dataset: il primo giorno di ogni cluster non ha un giorno precedente disponibile, quindi viene sempre scartato.

---

9. Problema del Persistence Shortcut e Soluzioni Sperimentate

Questa è la sfida principale incontrata durante lo sviluppo del Regression Learner.

Il Problema: Persistence Shortcut

Nel contesto della tabular regression per serie temporali ad alta autocorrelazione, emerge sistematicamente il seguente comportamento patologico:

Con `AAC_energy_t_1` nella matrice X:

Il consumo energetico è una serie altamente autocorrelata: il coefficiente di autocorrelazione al lag 1 (30 minuti) è tipicamente > 0.95. Ciò significa che `AAC_energy_t_1 ≈ AAC_energy_t_0` nella maggior parte dei casi.

Un ensemble di gradient boosting lo scopre immediatamente durante il training e trova la soluzione ottimale banale:

```
ŷ(t) ≈ AAC_energy_t_1 = y(t-1)
```

Il modello diventa matematicamente equivalente a un **modello di persistenza** (predict-previous), ma presenta metriche di training eccellenti perché effettivamente l'errore `|y(t) - y(t-1)|` è piccolo nella maggior parte degli step.

Visivamente, le predizioni appaiono come una copia "shiftata" di 30 minuti del segnale reale.

Soluzione 1 — Differencing (Sperimentata)

Idea: il modello viene allenato non sul valore assoluto `y(t)`, ma sulla **variazione** `Δy(t) = y(t) - y(t-1)`. Le predizioni vengono poi ricostruite come `ŷ(t) = Δŷ + y(t-1)`.

Implementazione:

```
yTrainPrev  = XTrain(:, indice_di_AAC_energy_t_1);
yTrainDelta = yTrain - yTrainPrev;
% Addestra su Delta
```

```
mdl = fitrensemble(XTrain, yTrainDelta, ...);
% Ricostruzione
yTestPred = predict(mdl, XTest) + yTestPrev;
```

Risultati (Zona 9, ensemble):

Metrica	Prima (con t-1)	Con Differencing
R^2 Test	0.6417	0.6706
RMSE Test	48.35 kWh	46.35 kWh

Problema residuo: Se il modello non riesce a capire la direzione del cambiamento, la previsione ottimale sotto MSE è `Δŷ ≈ 0` (la media del target di training è ≈ 0). Ma `Δŷ = 0 → ŷ(t) = y(t-1)` — si ripresenta la persistenza.

Soluzione 2 — Rimozione di `AAC_energy_t_1` (Sperimentata)

Idea: rimuovere solo la colonna lag-1 del target dalla matrice X, lasciando intatti i lag 2..48 e tutte le feature meteo/tempo.

Problema: il modello impara a usare `AAC_energy_t_2` come proxy di `y(t-1)`, poiché `AAC_energy_t_2 ≈ AAC_energy_t_1 ≈ y(t)` per alta autocorrelazione. Il comportamento di copia si ripresenta con un offset di 2 step.

Questa soluzione **non risolve** il problema strutturale: indipendentemente da quanti lag brevi di `AAC_energy` vengono rimossi, il modello troverà il lag più recente disponibile e lo userà come shortcut.

Soluzione 3 — Solo Feature Esogene (Approccio Attuale, `_exog`)

Idea: rimuovere completamente `AAC_energy` dalla lista dei predictors. Il modello vede **solo feature esogene** (meteo + tempo ciclico).

```
% Config attuale
predictors = ["precipprob", "temp", "windspeed", "holiday_indicator",
              "hour_sin", "hour_cos", "day_sin", "day_cos"];
% 8 feature × 48 lag = 384 colonne di input
```

Effetto: il modello non ha accesso a nessun valore storico del consumo. Deve predire da zero basandosi solo su:

- Che ora è (encoding ciclico)
- Che giorno della settimana è
- Che tempo fa (temperatura, vento, pioggia)
- Se è festivo

Risultati (Zona 9, ensemble, config exog):

Metrica	Config originale	Config exog
R^2 Train	0.9139	0.8678
R^2 Val	0.7895	0.5881
R^2 Test	0.6417	**0.6859**
RMSE Test	48.35 kWh	**45.27 kWh**

Osservazione sorprendente: il R^2 sul test set con la config exog (0.6859) è **superiore** alla config originale (0.6417), anche rimuovendo l'informazione autoregressive. Questo conferma che nella config originale il "guadagno" dovuto ad `AAC_energy_t_1` era fondamentalmente una **scorciatoia di copia**, non un apprendimento genuino del pattern.

---

10. Valutazione e Metriche

Metriche di Qualità

R^2 (Coefficiente di Determinazione)

```
R² = 1 - SS_res / SS_tot  =  1 - Σ(y - ŷ)² / Σ(y - ȳ)²
```

Varia in [0,1] in assenza di modelli peggiori della media. Indica la percentuale di varianza del target spiegata dal modello. È la metrica più informativa per comparare modelli su dataset diversi perché è adimensionale.

RMSE (Root Mean Square Error)

```
RMSE = √(mean((y - ŷ)²))    [kWh]
```

Nella stessa unità del target. Penalizza fortemente gli errori grandi (outlier). Ideale per comparare modelli sullo stesso test set.

MAE (Mean Absolute Error)

```
MAE = mean(|y - ŷ|)    [kWh]
```

Più robusto agli outlier rispetto a RMSE. Rappresenta l'errore medio tipico che ci si aspetta in un'operazione di previsione.

MAPE (Mean Absolute Percentage Error)

```
MAPE = mean(|y - ŷ| / |y|) × 100    [%]  (solo per |y| > 1e-3)
```

Metrica intrinsecamente instabile su questo dataset: nelle ore notturne e nei giorni festivi il consumo può essere vicino a zero, rendendo la divisione esplosiva. Nel codice viene calcolato solo per i campioni con `|y| > 1e-3 kWh`. I valori tipici (85-130%) riflettono questa instabilità e **non devono essere interpretati letteralmente**.

Baseline di Persistenza

La vera misura del valore del modello è il confronto con il **modello di persistenza**:

```
ŷ_pers(t) = y(t-1)    ← predici che il prossimo valore sia uguale al corrente
```

Questo modello è stupidamente semplice ma sorprendentemente efficace su serie temporali stazionarie a breve termine. Se un modello di machine learning non batte la persistenza, non aggiunge nessun valore.

Persistenza Zona 9 (test set):

- R^2 = 0.6080, RMSE = 50.79 kWh, MAE = 37.57 kWh

Analisi Diagnostica Implementata

Oltre alle metriche aggregate, `RegressionLearner.m` include 4 analisi diagnostiche per capire *dove* e *come* il modello sbaglia:

1. **Time-Series Plot** (`plotRegressionResults`): confronto visivo real vs predicted sul test set con separatori per giornata.
2. **Scatter Plot: Real vs Predicted** — idealmente i punti si allineano attorno alla bisettrice y=x.
3. **Residual Analysis** (4 pannelli):
   - Residui nel tempo: scatter degli errori su scala temporale
   - Distribuzione residui: istogramma (idealmente campana centrata in 0)
   - ACF dei residui: se autocorrelata → il modello ha perso pattern temporali
   - Q-Q Plot: verifica normalità dei residui
4. **Hourly Error Breakdown**: MAE medio per fascia oraria (0-23h) con barre di errore standard. Identifica le ore critiche.

---

11. Risultati Sperimentali Comparativi

Zona 9 — Trieste: Tutti gli Esperimenti

Configurazione Baseline (9 feature × 48 lag, `_orig`)

Configurazione identica all'LSTM in termini di feature, ma con approccio tabular.

Set	R^2	RMSE (kWh)	MAE (kWh)	MAPE (%)
Training	0.9139	35.44	25.78	56.57
Validation	0.7895	60.73	41.95	58.96
Test	**0.6417**	**48.35**	**35.54**	**117.78**
Persistenza	0.6080	50.79	37.57	—

Margine su persistenza: R^2 +0.034, RMSE -2.44 kWh (-4.8%)

Con Differencing (`_diff`)

Target = Δy = y(t) - y(t-1). Ricostruzione: ŷ(t) = Δŷ + y(t-1).

Set	R^2	RMSE (kWh)	MAE (kWh)	MAPE (%)
Training	0.9110	36.03	26.08	58.78
Validation	0.7871	61.08	43.09	68.91
Test	**0.6706**	**46.35**	**33.97**	**102.15**
Persistenza	0.6080	50.79	37.57	—

Margine su persistenza: R^2 +0.063, RMSE -4.44 kWh (-8.7%) — miglioramento rispetto alla baseline.

Solo Feature Esogene (`_exog`) — Configurazione Attuale

Target = y(t) diretto. Nessuna feature AAC_energy. 8 feature × 48 lag = 384 colonne.

Set	R^2	RMSE (kWh)	MAE (kWh)	MAPE (%)
Training	0.8678	43.91	32.09	85.54

Validation	0.5881	84.95	64.15	115.35
Test	**0.6859**	**45.27**	**34.75**	**130.68**
Persistenza	0.6080	50.79	37.57	—

Margine su persistenza: R^2 +0.078, RMSE -5.52 kWh (-10.9%) — **migliore in assoluto sul test set**.

> ***Interpretazione chiave:*** *Il fatto che la config exog abbia il test R^2 più alto (0.6859) nonostante la rimozione di* `AAC_energy` *conferma che nelle configurazioni precedenti l'informazione autoregressive contribuiva quasi esclusivamente attraverso il persistence shortcut, non attraverso un apprendimento genuino. Le previsioni exog sono più "oneste" e generalizzano meglio su giorni nuovi.*

Riepilogo Comparativo: Tutti i Modelli

Configurazione	R^2 Test	RMSE Test (kWh)	Δ vs Persistenza
Persistenza (baseline)	0.6080	50.79	—
Baseline (9 feature)	0.6417	48.35	+3.4% R^2, -4.8% RMSE
Con Differencing	0.6706	46.35	+6.3% R^2, -8.7% RMSE
Exog only (attuale)	**0.6859**	**45.27**	**+7.8% R^2, -10.9% RMSE**
LSTM (ref. `LSTM.m`)	~0.67–0.79	~47–61 kWh	dipende dal trial

Pattern degli Errori per Ora

In tutti gli esperimenti emerge un pattern sistematico:

- **Errore massimo:** fascia 11-14h (ore centrali della giornata, picco di attività)
- **Errore minimo:** fascia 3-4h (notte profonda, consumo quasi costante)

L'ora 11:00 mostra MAE tipicamente 3-6× superiore all'ora 04:00. Questo è un pattern universale nelle zone urbane residenziali-commerciali: il brusco cambio tra consumo notturno piatto e il picco mattutino/pomeridiano è influenzato dal comportamento umano (orari di lavoro, abitudini) che le feature meteo e cicliche catturano solo parzialmente.

---

12. Analisi Critica e Confronto con LSTM

Punti di Forza del Regression Learner

1. **Velocità di training:** pochi secondi vs decine di minuti per LSTM (300 alberi su 2064 campioni × 384 feature).
2. **Semplicità implementativa:** nessun layer, no gradient, no padding, no cell array.
3. **Feature importance:** `fitrensemble` fornisce direttamente l'importanza di ogni feature tramite `mdl.FeatureImportance`, utile per capire quali lag meteorologici sono più predittivi.
4. **Robustezza agli outlier:** LSBoost con alberi piccoli è naturalmente robusto a picchi anomali di consumo.

Limitazioni Rispetto all'LSTM

1. **Nessuna memoria implicita:** ogni riga del dataset è indipendente per il modello. L'LSTM mantiene invece uno stato interno che comprende implicitamente l'intera traiettoria temporale.

2. **Curse of dimensionality:** con 384-432 colonne di input altamente correlate e solo 2064 campioni di training, il rapporto feature/campioni è sfavorevole (circa 1:5). L'LSTM con le stesse informazioni lavora su vettori di 9 feature per timestep.
3. **Non cattura stagionalità giornaliera complessa:** l'LSTM in modalità Seq2Seq apprende il "ritmo" giornaliero come pattern sequenziale; il Regression Learner deve encodarlo interamente nelle colonne lag.
4. **Persistence shortcut strutturale:** come dimostrato sperimentalmente, il modello tabular trova sempre la scorciatoia autoregressive più breve disponibile. L'LSTM soffre meno di questo problema poiché la sua architettura è progettata per soppesare diversamente i contributi temporali.

Quando Preferire il Regression Learner

Nonostante le limitazioni, il Regression Learner rimane preferibile in questi scenari:

- **Dataset piccoli** dove l'LSTM non ha abbastanza dati per convergere stabilmente
- **Interpretabilità richiesta:** analisi feature importance, spiegazione delle predizioni
- **Prototipazione rapida:** validare rapidamente se una combinazione di feature è predittiva
- **Sistemi embedded** dove un albero decisionale è più facilmente deployabile di una rete neurale

---

13. Possibili Sviluppi Futuri

Approccio A — Lag Stagionali Selezionati

Invece di usare tutti i lag 1..48 di `AAC_energy`, tenere solo i lag con significato fisico:

- `AAC_energy_t_48` → stesso istante 24 ore fa
- `AAC_energy_t_96` → stesso istante 48 ore fa (richiede `numLags = 96`)

Questo mantiene l'informazione autoregressive semanticamente significativa ed elimina la zona di "copia facile" dei lag brevi.

```
% Esempio: rimuovere AAC_energy_t_1 .. AAC_energy_t_47
lagToRemove = "AAC_energy_t_" + string(1:47);
% mantenere solo AAC_energy_t_48
```

Approccio C — Feature Engineering con Rolling Statistics

Sostituire i lag grezzi di `AAC_energy` con statistiche aggregate:

- `mean_AAC_1h` = media ultime 2 rilevazioni (= t-2, t-3 per evitare t-1)
- `mean_AAC_6h` = media ultime 12 rilevazioni
- `std_AAC_6h` = variabilità nelle ultime 6h
- `AAC_t_48` = stesso momento ieri
- `delta_24h` = AAC_t_48 - AAC_t_96 (trend rispetto a 2 giorni fa)

Questo approccio, usato in produzione nei sistemi di forecasting industriali, cattura il contesto senza esporre il valore esatto di t-1.

Ottimizzazione Iperparametri

Implementare una ricerca Bayesiana degli iperparametri dell'ensemble analoga a quella in `LSTM.m`, con objective = validation RMSE:

```
optimVars = [
    optimizableVariable("NumLearningCycles", [50, 500], "Type", "integer")
    optimizableVariable("LearnRate",         [0.01, 0.3], "Transform", "log")
    optimizableVariable("MaxNumSplits",      [2, 20],   "Type", "integer")
];
```

Previsione Multi-Step

Estendere da previsione one-step-ahead (t+1) a previsione multi-step (es. t+8 = 4 ore), creando 8 modelli indipendenti (uno per ogni orizzonte) o usando un modello ricorsivo che usa le proprie predizioni come input.

Confronto Multi-Zona

Addestrare il Regression Learner exog su una zona e valutarlo sulle altre 3, analogo all'esperimento cross-zona eseguito con l'LSTM, per verificare se le feature esogene generalizzano tra zone urbane con diverso carattere.

Fine del documento. Ultima revisione: Marzo 2026.